



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

КУРСОВАЯ РАБОТА

по дисциплине

«Технологии обработки и организации хранения статистических данных»

Тема курсовой работы: «Сравнительный анализ рынка компьютерных игр в России и мире»

Студент группы ИНБО-20-23

Боргачев Тимофей Максимович

(подпись)

Руководитель
курсовой работы

к.т.н., доцент кафедры ПМ,
Щербань П.С.

(подпись)

Работа представлена к защите

«__» _____ 2024 г.

Допущен к защите

«__» _____ 2024 г.

Москва 2024 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

Утверждаю

Заведующий кафедрой ПМ

 Смоленцева Т.Е.
(подпись)

«20» сентября 2024 г.

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Технологии обработки и организации хранения статистических данных»

Студент Боргачев Тимофей Максимович

Группа ИНБО-20-23

Тема «Сравнительный анализ рынка компьютерных игр в России и мире»

Исходные данные: собранный студентом набор данных по теме работы

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

Характеристика изучаемой предметной области, алгоритма, набора данных (описание текущего состояния исследуемой предметной области, выделение перспективных направлений исследований, применимость алгоритмов анализа и обработки данных, описание полей набора данных)

Математическая формулировка предлагаемого метода анализа и обработки данных (классическая постановка задачи, формулировка задачи статистической обработки данных, описание параметров, описание критерия качества решения конечной задачи)

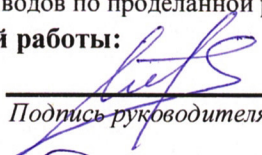
Анализ полученной выборки данных с использованием предложенных методов анализа и обработки данных (описание последовательности действий или сценария обработки данных)

Построение визуализаций и качественных выводов по проделанной работе

Срок представления к защите курсовой работы:

до «16» декабря 2024 г.

Задание на курсовую работу выдал


Подпись руководителя

Щербань П.С.

(ФИО руководителя)

«20» сентября 2024 г.

Задание на курсовую работу получил


Подпись обучающегося

Боргачев Т.М.

(ФИО обучающегося)

«20» сентября 2024 г.

ОТЗЫВ
на курсовую работу
по дисциплине «Технологии организации обработки и хранения
статистических данных»

Студент(ка)

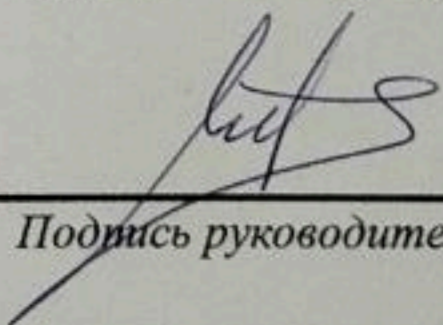
Боргаев П.М

ИНБО-20-23

Характеристика курсовой работы

Критерий	Да	Нет	Не полностью
1. Соответствие содержания курсовой работы указанной теме	✓		
2. Соответствие курсовой работы заданию	✓		
3. Соответствие рекомендациям по оформлению текста, таблиц, рисунков и пр.	✓		
3. Полнота выполнения всех пунктов задания	✓		
4. Логичность и системность содержания курсовой работы	✓		
5. Отсутствие фактических грубых ошибок	✓		

Рекомендуемая оценка: удовлетворительно, хорошо, отлично
(нужное подчеркнуть)


Подпись руководителя

Щербань П. С.
(ФИО руководителя)

«19» ноября 2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1 Регрессионный анализ.....	5
1.1.1 Линейная регрессия.....	5
1.1.2 Полиномиальная регрессия	9
1.1.3 Логистическая регрессия	10
1.1.4 Метрики для оценки регрессии	10
1.1.5 Недостатки классических регрессионных моделей	12
1.2 Нейросети	13
1.2.1 Введение в нейросети.....	13
1.2.2 Регрессионный анализ с помощью нейросетей	15
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	20
2.1 Предобработка данных	20
2.2 Корреляционный анализ	26
2.3 Обучение моделей.....	27
ЗАКЛЮЧЕНИЕ.....	36
ИНФОРМАЦИОННЫЕ ИСТОЧНИКИ.....	37
Теоретическая часть.....	37
Практическая часть	37
ПРИЛОЖЕНИЕ.....	38
Приложение А	39

ВВЕДЕНИЕ

Задача линейной регрессии является одним из наиболее известных и простых методов машинного обучения, который широко используется для решения задач прогнозирования и анализа данных.

Классическая линейная регрессия решается с помощью методов минимизации ошибки, таких как метод наименьших квадратов, и обычно применяется для небольших и средних наборов данных, где связь между переменными выражена линейно.

В последние десятилетия значительное внимание исследователей привлекли нейросети, которые предлагают более гибкие и мощные инструменты для моделирования зависимостей в данных. Одним из ключевых преимуществ нейросетевых моделей является их способность обнаружить как линейные, так и нелинейные функции в качестве зависимости, что делает их применимыми к более сложным данным. Использование нейронных сетей для решения задачи регрессии, в том числе линейной, позволяет улучшить точность прогнозов и учитывать более сложные зависимости между переменными.

Цель данной курсовой работы — исследовать задачу линейной регрессии, реализованную как в её классической форме, так и с использованием нейросетей. В работе будут рассмотрены теоретические аспекты обоих подходов, их преимущества и недостатки, а также представлены результаты их применения на реальных данных.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Регрессионный анализ

1.1.1 Линейная регрессия

Одним из основных инструментов анализа данных является регрессионный анализ. В начале XIX века практически одновременно двумя учеными-математиками А.М. Лежандром и К.Ф. Гауссом был изобретён метод наименьших квадратов, который вызвал к жизни регрессионный анализ.

Метод наименьших квадратов является математическим принципом: при его использовании определяют условия, при которых значения квадрата некоторой функции F получаются наименьшими:

$$F^2 = \min.$$

В таком общем виде принцип наименьших квадратов находит широкое применение при математическом описании научных и технических задач, например, в физике, механике, математической статистике, в теории ошибок при обработке экспериментальных данных, при решении задач численного анализа.

Термин «регрессия» (от лат. «regressio» - движение назад) был введен Ф. Гальтоном, когда он, изучая зависимость между ростом родителей и их детей, обнаружил явление «регрессии к среднему» - у детей, родившихся у очень высоких родителей, рост имел тенденцию быть ближе к средней величине. По данным о росте 205 пар родителей и 928 их взрослых детей, Ф. Гальтон оценил неизвестный параметр a следующей приближенной зависимостью:

$$y_i \approx \bar{y} + \alpha(x_i - \bar{x}),$$

где y_i , – рост i – го ребёнка; x_i , – средний рост его родителей.

Таким образом рост человека стремится к среднему, не увеличивается и не уменьшается, то есть отсутствует прогресс. А отсутствие прогресса – это регресс. Отсюда возник термин «регрессионный анализ».

С появлением вычислительной техники развитие алгоритмов регрессионного анализа воистину было путём «вверх по лестнице, ведущей вниз». С каждым новым поколением электронно-вычислительных машин рождались новые алгоритмы: метод всех возможных регрессий, шаговый метод, ступенчатый метод и другие.

Но всякий раз решение конкретных задач методами регрессионного анализа давало неоднозначные, разные и множественные ответы на поставленные вопросы. Математики начали относить регрессионные задачи к классу некорректно поставленных, а сам анализ в их понимании деградировал до чисто эвристического метода, и автоматизация его задач потому зашла в тупик. Однако параллельно шёл другой процесс, который поставил регрессионный анализ в центр проблематики многомерной статистики и связал его с дисперсионным, дискриминантным, кластерным, факторным анализами. Регрессионный анализ оказался одним из «столпов» планирования экспериментов. Его обобщили для решения широких классов задач нелинейной параметризации, начиная с задач химической кинетики и заканчивая построением динамических моделей идентификации и управления производством. Он получил многочисленные эконометрические приложения, связь со спектральным анализом и теорией фильтрации, с решением уравнений математической физики и так далее.

Регрессионный анализ представляет собой методику, которая направлена на установление зависимости между зависимой переменной (отклик) и одной или несколькими независимыми переменными (предикторы). Эта техника широко используется для прогнозирования и объяснения поведения сложных систем в различных областях знаний, включая экономику, инженерию, медицину, биологию и многие другие.

Наиболее простой и часто используемой моделью регрессионного анализа является линейная регрессия. Она предполагает, что зависимость между переменными можно описать линейной функцией вида:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon,$$

здесь y – зависимая переменная, $x_1, x_2, \dots, x_n$ – независимые переменные, $\beta_0, \beta_1, \dots, \beta_n$ – коэффициенты регрессии, а ε – случайная ошибка

Метод наименьших квадратов позволяет минимизировать сумму квадратов ошибок и найти наилучшие оценки для коэффициентов регрессии.

Данный метод является одним из базовых подходов в регрессионном анализе. Он предполагает минимизацию суммы квадратов отклонений наблюдаемых значений от предсказанных моделью. Допустим, у нас есть набор данных: $\{(x_i, y_i)\}_{i=1}^n$, где x_i — независимые переменные, а y_i — зависимые переменные. Тогда наша задача заключается в минимизации следующей функции:

$$L(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}))^2$$

Где $\beta_0, \beta_1, \dots, \beta_p$ – коэффициенты регрессии. Этот метод является эффективным при условии, что данные подчиняются гауссовскому распределению ошибок, и все независимые переменные имеют линейную зависимость с откликом.

В некоторых случаях зависимости между переменными не могут быть описаны линейной моделью. В таких случаях используется нелинейная регрессия, которая предполагает более сложные модели, такие как полиномиальные, логарифмические или экспоненциальные зависимости. Примером нелинейной регрессии может служить логистическая регрессия, которая применяется для моделирования вероятностей бинарных исходов.

Множественная регрессия расширяет линейную регрессию на случай нескольких предикторов. Это позволяет моделировать системы, где отклик зависит от более чем одного фактора. Например, в экономике доход человека может зависеть не только от его образования, но и от возраста, стажа работы и других факторов.

Регрессионный анализ находит свое применение в различных областях:

- эконометрика — изучение макро- и микроэкономических процессов;
- медицина — прогнозирование заболеваний на основе клинических данных;
- биология — исследование зависимостей между параметрами популяций;
- инженерия — моделирование физических процессов, таких как теплопередача или механическое напряжение.

Одним из главных ограничений регрессионного анализа является его чувствительность к качеству данных. Модели регрессии могут давать неточные прогнозы при наличии выбросов, отсутствующих данных или мультиколлинеарности (сильная корреляция между независимыми переменными). Кроме того, регрессионные модели могут быть переобучены, что приводит к плохим результатам на новых данных. Для решения этих проблем применяются такие методы, как регуляризация (Ridge, Lasso), кросс-валидация и стандартизация данных.

Рассмотрим подробнее варианты регуляризации:

- ridge-регрессия: метод, который добавляет штраф за большие значения коэффициентов регрессии. Функция потерь дополняется регуляризатором, что выглядит так:

$$L(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2,$$

где λ — коэффициент регуляризации, контролирующий степень штрафа за большие значения β_j ;

- lasso-регрессия: метод, который также накладывает штраф на значения коэффициентов, но вместо суммы квадратов используется сумма модулей коэффициентов:

$$L(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|,$$

этот метод способен обнулить некоторые коэффициенты, что ведет к получению "разреженной" модели, где только значимые переменные имеют ненулевые коэффициенты.

1.1.2 Полиномиальная регрессия

Полиномиальная регрессия является расширением линейной регрессии, в которой зависимая переменная моделируется как полиномиальная функция независимых переменных. В отличие от линейной регрессии, где зависимость между переменными является прямолинейной, полиномиальная регрессия позволяет учесть более сложные нелинейные зависимости. Уравнение полиномиальной регрессии для одной переменной может быть записано как:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n + \varepsilon$$

Применение полиномиальной регрессии позволяет моделировать более сложные зависимости между переменными, которые не удастся уловить с помощью линейной модели.

Несмотря на свою гибкость, полиномиальная регрессия имеет и свои недостатки:

- переобучение: при большом числе степеней полинома модель может начать запоминать шум в данных, что ухудшит её способность к обобщению на новых данных;
- экспоненциальный рост сложности: с увеличением степени полинома количество коэффициентов и вычислительная сложность существенно возрастают;
- интерпретируемость: коэффициенты полинома имеют менее интуитивно понятное значение по сравнению с линейной регрессией.

1.1.3 Логистическая регрессия

Логистическая регрессия является важным методом для работы с бинарными данными, когда зависимая переменная принимает значения 0 или 1. Модель логистической регрессии может быть записана следующим образом:

$$p(y = 1 | x) = \frac{1}{1 + e - (\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}$$

Здесь $p(y = 1 | x)$ — вероятность того, что $y = 1$, а $\beta_0, \beta_1, \dots, \beta_n$ — параметры модели. Логистическая регрессия активно используется в задачах классификации, таких как распознавание образов, медицина и кредитный скоринг.

В большинстве случаев логистическая регрессия применяется для решения задач классификации.

При наличии большого количества конечных классов используют стратегию «один против всех», когда каждый класс противопоставляется остальным, чтобы не нарушать бинарности функции на выходе логистической регрессии.

1.1.4 Метрики для оценки регрессии

После обучения модели для предсказаний встает вопрос об оценке модели, для понимания – хорошо она предсказывает или нет.

Это отдельная непростая тема в задачах регрессии, потому что нужно учитывать не только точность модели – насколько ее предсказания близки к настоящим значениям, но также и переобучение модели, возможная смена поведения функции.

Когда модель отлично показывает себя на тестовых данных, нельзя сделать вывод о том, что она так же справится и с реальными данными, что будет свидетельствовать о переобучении.

Для оценки качества регрессионных моделей используются различные метрики, которые позволяют измерить, насколько предсказания модели совпадают с реальными данными. Основные метрики для регрессии включают:

1. Коэффициент детерминации R^2 :

Коэффициент детерминации R^2 показывает, какая доля дисперсии зависимой переменной объясняется независимыми переменными. Он рассчитывается следующим образом:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

Где y_i – реальные значения, $\hat{y}_i$ – предсказанные значения, $\bar{y}_i$ – среднее значение зависимой переменной.

Значение R^2 может варьироваться от 0 до 1. Если $R^2 = 1$, это означает, что модель идеально предсказывает данные. Если $R^2 = 0$, это означает, что модель вообще не объясняет дисперсию зависимой переменной.

2. Среднеквадратичная ошибка:

Среднеквадратичная ошибка показывает среднее значение квадрата отклонений предсказанных значений от реальных:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE измеряется в тех же единицах, что и зависимая переменная, но в квадрате, поэтому иногда труден для интерпретации.

3. Средняя абсолютная ошибка:

Средняя абсолютная ошибка показывает среднее значение абсолютных отклонений предсказанных значений от реальных:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE проще интерпретировать, чем MSE, так как измеряется в тех же единицах, что и зависимая переменная, и не подвержен влиянию больших выбросов.

4. Корень из среднеквадратичной ошибки:

Среднеквадратичная ошибка с регуляризацией представляет собой квадратный корень из среднеквадратичной ошибки. Это делает её более интерпретируемой, чем MSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE часто используется для сравнения точности различных моделей, так как она чувствительна к большим отклонениям.

1.1.5 Недостатки классических регрессионных моделей

Классические регрессионные модели обладают рядом ограничений. В частности:

- линейность: во многих реальных приложениях зависимость между переменными является нелинейной, и это приводит к необходимости использования более сложных моделей, таких как нейросети;
- мультиколлинеарность: высокая корреляция между независимыми переменными может привести к нестабильности оценок коэффициентов;
- неправильная спецификация модели: если модель неверно описывает структуру данных, это может привести к смещенным или неэффективным оценкам параметров.

1.2 Нейросети

1.2.1 Введение в нейросети

Нейросети, или искусственные нейронные сети, — это вычислительные модели, вдохновленные работой биологических нейронных сетей. Они активно используются для решения задач в области машинного обучения и искусственного интеллекта, таких как распознавание образов, прогнозирование временных рядов, обработка естественного языка и других.

Основной элемент нейронной сети — это искусственный нейрон, который имитирует поведение биологического нейрона. Он принимает на вход несколько сигналов, умножает их на соответствующие веса, затем складывает результаты и применяет к ним активационную функцию. В результате на выходе нейрона получается сигнал, который передается другим нейронам в сети.

Искусственные нейронные сети состоят из следующих компонентов:

1. Входной слой — получает исходные данные.
2. Скрытые слои — выполняют нелинейное преобразование данных. В многослойных персептронах таких слоев может быть несколько.
3. Выходной слой — генерирует финальный результат, например, вероятность принадлежности объекта к определенному классу.

Нейронные сети могут быть различными по своей структуре и назначению:

1. Многослойный персептрон (MLP) — это классическая полносвязная нейронная сеть, где каждый нейрон одного слоя связан с каждым нейроном следующего слоя.
2. Сверточные нейронные сети (CNN) — используются для обработки изображений. Они включают в себя сверточные слои, которые позволяют выявлять важные признаки изображения, такие как края и текстуры.

3. Рекуррентные нейронные сети (RNN) — применяются для анализа последовательных данных, таких как временные ряды или текст. В таких сетях нейроны могут передавать информацию не только в следующий слой, но и "назад", что позволяет учитывать контекст предыдущих данных.
4. Генеративные состязательные сети (GAN) — состоят из двух частей: генератора и дискриминатора, которые "соревнуются" друг с другом, что позволяет генератору создавать новые данные, очень похожие на исходные (например, фальшивые изображения или видео).

Для придания модели нейросети нелинейных свойств используются активационные функции. Некоторые из популярных функций:

- **ReLU** - Rectified Linear Unit — активируется, если входное значение больше нуля, иначе возвращает ноль:

$$ReLU(x) = \max(0, x)$$

ReLU проста в вычислении и помогает сети обучаться быстрее за счет уменьшения эффекта исчезающего градиента. Однако она может привести к тому, что некоторые нейроны "выключаются" навсегда (градиент становится равным нулю);

- **leaky ReLU**: модификация ReLU, которая решает проблему "мертвых нейронов", позволяя небольшие отрицательные значения:

$$LeakyReLU(x) = \max(\alpha x, x),$$

где α — маленькое положительное число;

- **sigmoid** — сжимает значения к диапазону $[0, 1]$, используется для моделирования вероятностей:

$$Sigmoid(x) = \frac{1}{1 + e^{-x}}$$

однако у неё есть недостаток — она страдает от эффекта исчезающего градиента;

- $\tanh$ — возвращает значения в диапазоне $[-1, 1]$, используется в рекуррентных сетях для стабилизации градиентов:

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

это особенно полезно в нейронных сетях, где сигналы могут быть как положительными, так и отрицательными.

Процесс обучения нейронной сети заключается в нахождении оптимальных весов с помощью алгоритма обратного распространения ошибки и метода градиентного спуска. Алгоритм итеративно минимизирует функцию ошибки, корректируя веса нейронов, чтобы улучшить точность предсказаний.

Нейронные сети нашли применение в самых различных сферах:

- компьютерное зрение — задачи распознавания лиц, классификация изображений, анализ видео;
- обработка естественного языка — машинный перевод, чат-боты, генерация текста;
- здравоохранение — диагностика заболеваний по медицинским изображениям (рентген, МРТ);
- автономные системы — управление беспилотными транспортными средствами, дронами и роботами.

1.2.2 Регрессионный анализ с помощью нейросетей

Нейросетевые методы регрессии становятся все более популярными благодаря их способности моделировать сложные нелинейные зависимости, которые классические методы регрессии не могут уловить. В отличие от линейных и полиномиальных моделей, нейронные сети могут адаптироваться к практически любым типам данных благодаря своей архитектуре.

Полносвязная нейронная сеть — это один из самых базовых вариантов нейронных сетей, который часто используется для задач регрессии. В этой сети каждый нейрон одного слоя связан с каждым нейроном следующего слоя. Скрытые слои применяют нелинейные активационные функции, что позволяет сети моделировать сложные зависимости между переменными.

Примером архитектуры для задачи регрессии может быть сеть, состоящая из:

1. Входного слоя с числом нейронов, равным количеству предикторов.
2. Нескольких скрытых слоев с активационной функцией ReLU.
3. Выходного слоя с одним нейроном, который выдает предсказанное значение.

Хотя сверточные нейронные сети в основном применяются для анализа изображений, их также можно использовать для задач регрессии. Например, они могут применяться в задачах прогнозирования временных рядов, где сверточные слои позволяют выявлять важные временные паттерны в данных.

Рекуррентные нейронные сети (RNN) и их более сложные варианты, такие как долгосрочная краткосрочная память (LSTM), часто применяются для решения задач регрессии с временными рядами. Такие сети могут учитывать временную структуру данных, что делает их полезными для задач, где предсказания зависят от предыдущих наблюдений, например, в финансовом прогнозировании или в анализе спроса.

Для оценки регрессии на основе нейросетей, можно использовать все те же метрики, что и для классических вариантов регрессии. Но, помимо этого, существуют так же иные метрики:

1. Log-Cosh Loss

Log-Cosh Loss – это функция потерь, которая находится между MSE и MAE по чувствительности к выбросам. Она вычисляется как логарифм гиперболического косинуса разницы между предсказанным и реальным значением:

$$L(y, \hat{y}) = \sum_{i=1}^n \log(\cosh(y_i - \hat{y}_i))$$

Эта функция помогает смягчить влияние выбросов, поскольку она меньше штрафует большие ошибки по сравнению с MSE.

2. Huber Loss

Huber Loss – это комбинация MSE и MAE, которая в зависимости от величины ошибки применяет либо квадратичное, либо линейное наказание. Это полезно, когда данные содержат выбросы:

$$L(y, \hat{y}) = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2, & \text{если } |y_i - \hat{y}_i| \leq \delta \\ \delta \left(|y_i - \hat{y}_i| - \frac{1}{2}\delta \right), & \text{если } |y_i - \hat{y}_i| > \delta \end{cases}$$

Здесь δ - пороговое значение, которое определяет, когда нужно переключаться между MSE и MAE.

Процесс обучения нейросетей для задач регрессии схож с обучением для задач классификации, однако есть несколько важных отличий. Основная цель заключается в минимизации функции потерь, которая обычно измеряется с помощью метрик, таких как MSE, MAE или RMSE, описанные выше.

Алгоритм обучения включает следующие шаги:

1. Прямое распространение (Forward Propagation): на вход сети подаются данные, и они проходят через каждый слой сети для получения предсказаний.
2. Вычисление функции потерь: на основе разницы между предсказанными и реальными значениями вычисляется ошибка (например, MSE).
3. Обратное распространение (Backpropagation): вычисляется градиент ошибки по отношению к весам сети с использованием метода обратного распространения ошибки. Градиенты затем используются для обновления весов с целью минимизации ошибки.
4. Градиентный спуск (Gradient Descent): веса сети обновляются с шагом, определяемым скоростью обучения, чтобы минимизировать функцию потерь.

Регуляризация в нейросетях необходима для борьбы с переобучением, когда модель слишком точно запоминает обучающие данные и плохо обобщает на новые данные. Существует несколько методов регуляризации:

- dropout: это техника, которая заключается в случайном отключении части нейронов на каждом шаге обучения. Это помогает предотвратить зависимость сети от конкретных нейронов и улучшает её способность к обобщению;
- l2-регуляризация (Ridge): добавляет штраф за слишком большие веса в сети, что предотвращает их излишний рост. Это помогает сети оставаться "более гладкой" и не слишком подстраиваться под обучающие данные;
- ранняя остановка (Early Stopping): процесс, при котором обучение останавливается, если ошибка на валидационной выборке перестает уменьшаться. Это предотвращает переобучение модели на данных обучения.

Сравнение регрессионных методов на основе классических подходов и нейросетей может быть выполнено с нескольких точек зрения:

1. Способность к моделированию сложных зависимостей:
 - классическая регрессия: способна моделировать только те зависимости, которые явно задаются в виде линейных или полиномиальных уравнений;
 - нейросети: могут моделировать сложные, нелинейные зависимости благодаря использованию нескольких скрытых слоев и нелинейных активационных функций.
2. Чувствительность к мультиколлинеарности:
 - классическая регрессия: мультиколлинеарность может существенно ухудшать качество модели, так как затрудняет оценку параметров;
 - нейросети: не так сильно подвержены проблемам мультиколлинеарности, так как каждая нейронная связь обучается отдельно, что позволяет "обходить" коррелированные данные.

3. Гибкость модели:

- классическая регрессия: ограничена жесткими предположениями о линейности и нормальности распределений;
- нейросети: не требуют предположений о типе зависимости между переменными и могут работать с любыми типами данных (картинки, текст, временные ряды и др.).

4. Переобучение:

- классическая регрессия: риск переобучения возникает при наличии слишком большого числа предикторов по сравнению с количеством данных;
- нейросети: имеют тенденцию к переобучению при недостатке данных и избыточной сложности модели, но это можно контролировать с помощью методов регуляризации (например, dropout, L2-регуляризация).

5. Интерпретируемость:

- классическая регрессия: относительно легко интерпретировать результаты регрессии, так как каждый коэффициент имеет понятное значение (например, насколько изменится отклик при изменении предиктора на единицу);
- нейросети: являются "черными ящиками", и интерпретировать влияние отдельных переменных на результат крайне сложно.

Таким образом, нейросетевые методы регрессии обладают большей гибкостью и способностью моделировать сложные зависимости, но требуют больших вычислительных ресурсов и более осторожного подхода к обучению, чтобы избежать переобучения.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Предобработка данных

В соответствие с темой работы проведем регрессионный сравнительный анализ рынка видеоигр в России и мире. Для этого прежде всего необходимо ознакомиться с данными, визуализировать их и предобработать.

В датасете “Videogames_Sale_Data” представлено 10 столбцов и 7112 строк. Рассмотрим подробнее каждый из столбцов:

1. Platform – это видеоигровая платформа, для которой была выпущена данная видеоигра.
2. Genre – жанр видеоигры.
3. Publisher – издатель видеоигры.
4. NA_Sales – количество продаж видеоигры в США и Канаде.
5. EU_Sales – количество продаж видеоигры в Европе.
6. JP_Sales – количество продаж видеоигры в Японии.
7. Russia_Sales – количество продаж видеоигры в России.
8. Global_Sales – количество продаж в мире.
9. Rating – возрастной рейтинг видеоигры.
10. Critic_Score_Class – класс оценки игры критиками.

Продажи видеоигр, сгруппированные по жанрам и отсортированные по регионам представлены на Рисунке 2.1.

Продажи по жанрам и регионам

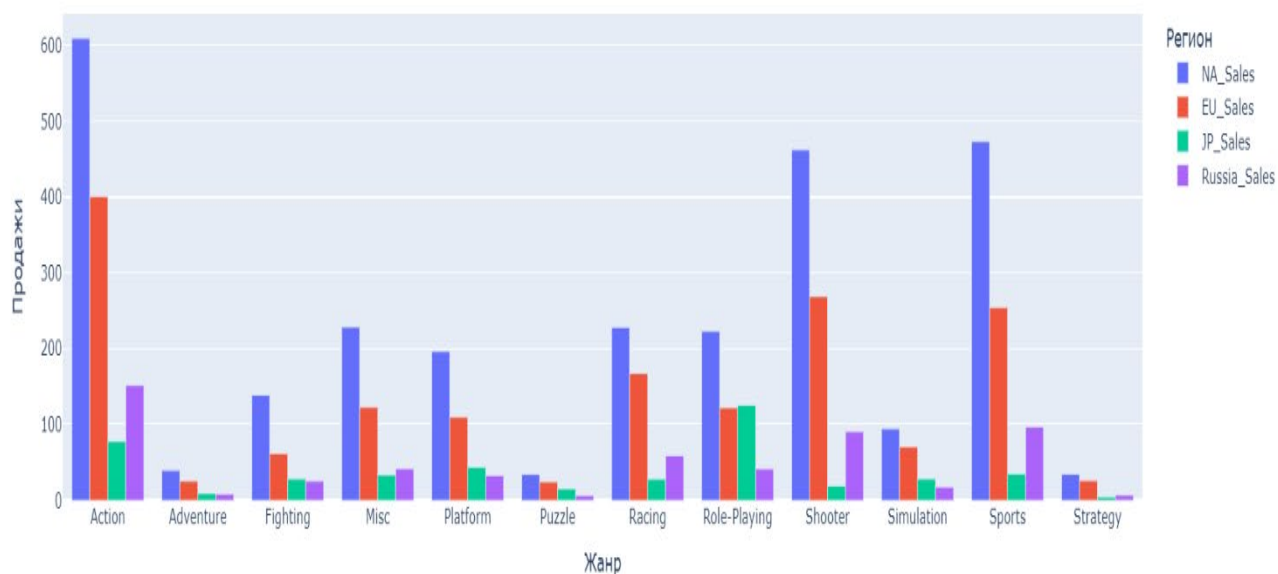


Рисунок 2.1 – Столбчатая диаграмма продаж видеоигр в разных регионах по жанрам

Рассмотрим жанры подробнее, отобразив их соотношение с помощью круговой диаграммы на Рисунке 2.2.

Распределение продаж по жанрам

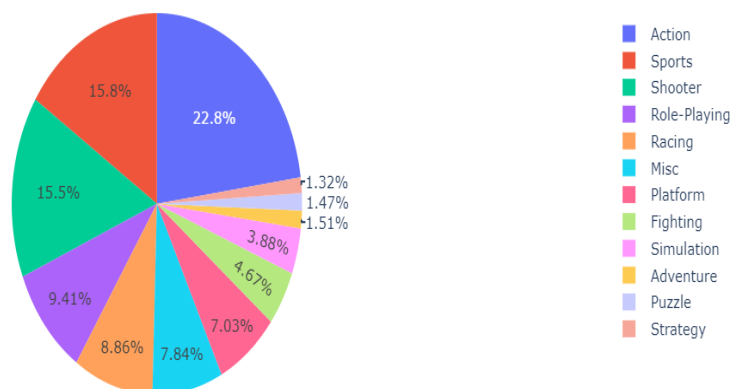


Рисунок 2.2 – Круговая диаграмма распределения продаж по жанрам

Отсортированные платформы по количеству продаж представлены на Рисунке 2.3.

Топ-10 платформ по глобальным продажам

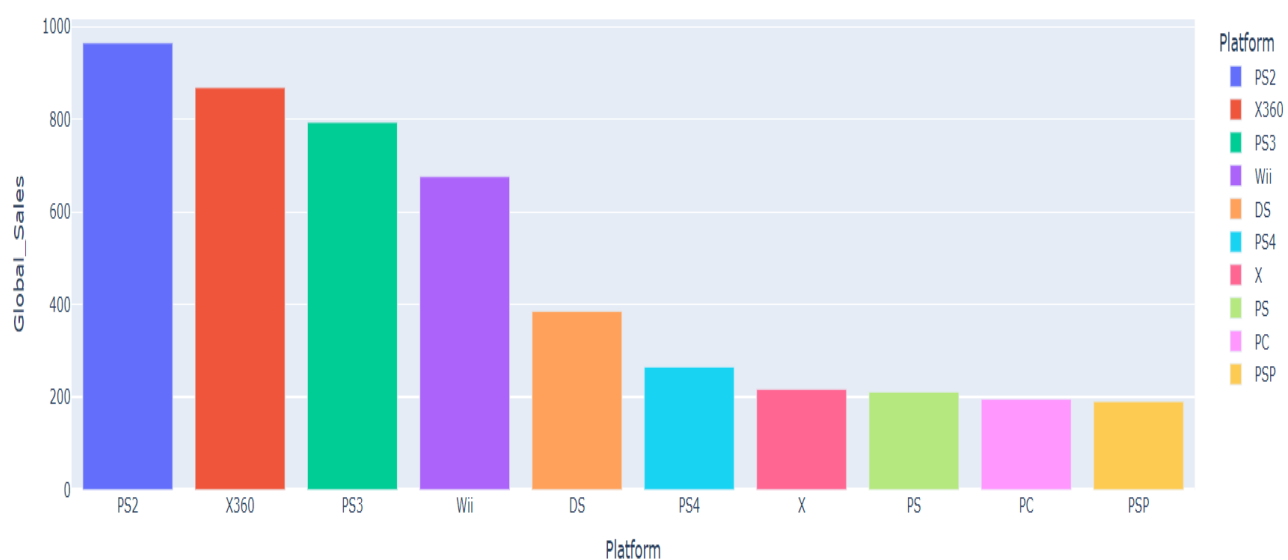


Рисунок 2.3 – Гистограмма платформ по продажам

Самые крупные издатели по глобальным продажам представлены на Рисунке 2.4.

Глобальные продажи видеоигр по топ-10 издателям

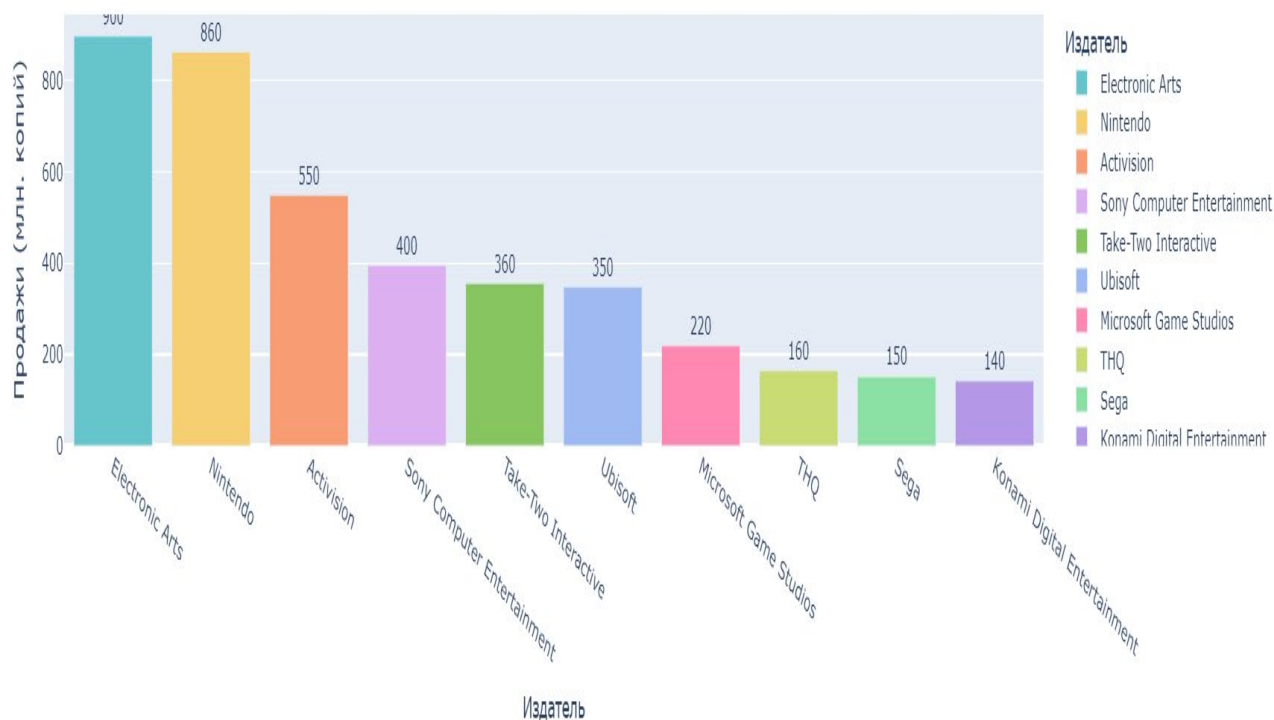


Рисунок 2.4 – Гистограмма издателей по глобальным продажам

Перейдем к предобработке, для этого первоначально посмотрим на количество пропусков и обработаем их.

Воспользуемся для этого встроенным в Logiном компонент «Заполнение пропусков».

Параметры заполнения показаны на Рисунке 2.5.

Заполнение пропусков

Исходные данные упорядочены

Допустимый процент пропусков

0

Random seed

937222705

№	Входные поля	Вид данных		Метод обработки
	Фильтрация			
1	ab Platform	Дискретный		Заменять наиболее вероятным
2	ab Genre	Дискретный		Заменять наиболее вероятным
3	ab Publisher	Дискретный		Заменять наиболее вероятным
4	90 NA_Sales	Непрерывный		Заменять медианой
5	90 EU_Sales	Непрерывный		Заменять медианой
6	90 JP_Sales	Непрерывный		Заменять медианой
7	90 Russia_Sales	Непрерывный		Заменять медианой
8	90 Global_Sales	Непрерывный		Заменять медианой
9	ab Rating	Дискретный		Заменять наиболее вероятным
10	ab Critic_Score_Class	Дискретный		Заменять наиболее вероятным

Рисунок 2.5 – Параметры заполнения пропусков

Далее избавимся от записей дубликатов, воспользуемся для этого компонентом «Дубликаты и противоречия» в совокупности с фильтром. Как видно из Рисунок 2.6 – было отброшено 48 записей дубликатов.

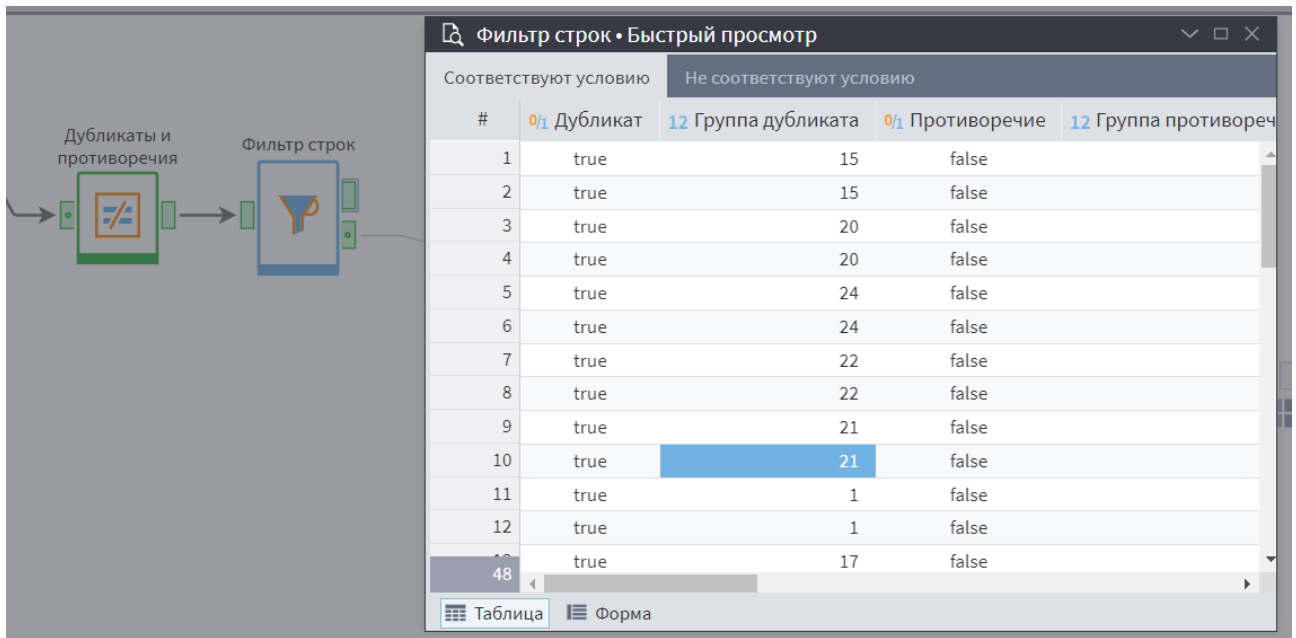


Рисунок 2.6 – Удаление дубликатов

Прежде чем перейти к обработке выбросов, построить boxplot – график распределения всех продаж. Как видно из Рисунка 2.7 выбросов очень много, распределение далеко от нормального.

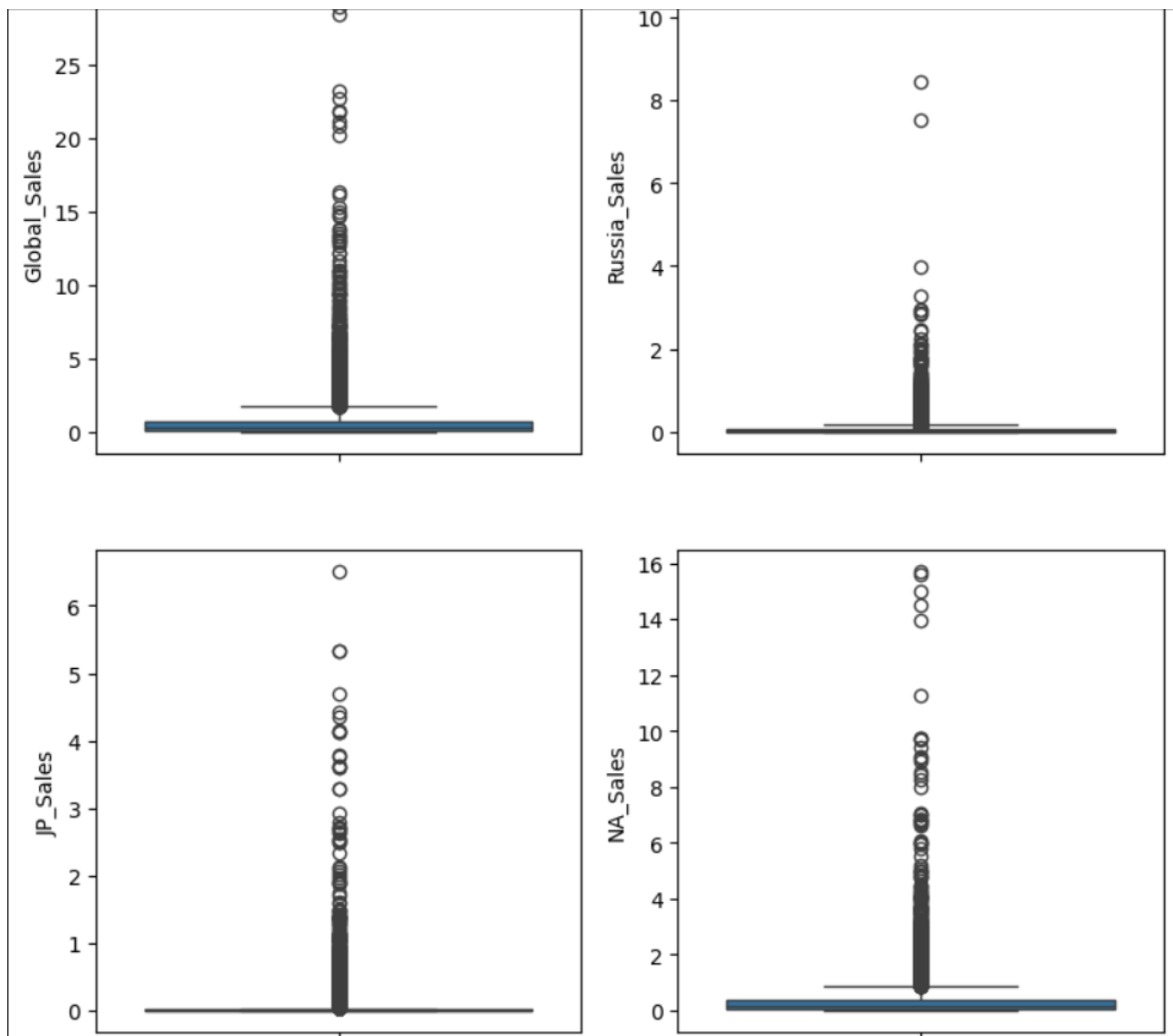


Рисунок 2.7 – Графики распределения численных признаков

Обработаем признаки “Russia_Sales” и “Global_Sales” с помощью компонента «Редактирование выбросов», заменив выбросы и экстремальные значения на наиболее вероятные значения, как показано на Рисунке 2.8.

Редактирование выбросов

Исходные данные упорядочены

Входные поля	Вид данных
Фильтрация	
Platform	дискретный
Genre	Дискретный
Publisher	Дискретный
NA_Sales	Непрерывный
EU_Sales	Непрерывный
JP_Sales	Непрерывный
Rating	Дискретный
Critic_Score_Class	Дискретный
Russia_Sales	Непрерывный

Определение выбросов и экстремальных значений

Метод выявления

Стандартное отклонение

Интерквартильная ширина

Для выброса

3

Для экстремального

5

Метод обработки выбросов

Заменять наиболее вероятным

Заменять на

0

Метод обработки экстремальных значений

Заменять наиболее вероятным

Заменять на

0

Рисунок 2.8 – Редактирование выбросов

2.2 Корреляционный анализ

Проведем корреляционный анализ отдельно дискретных и отдельно численных признаков для определения зависимостей по отношению к признакам, которые мы собираемся прогнозировать, а именно “Global_Sales” и “Russia_Sales”. Для дискретных признаков будем использовать коэффициент Спирмэна, а для численных коэффициент Пирсона.

Тепловые карты корреляции представлены на Рисунке 2.9.

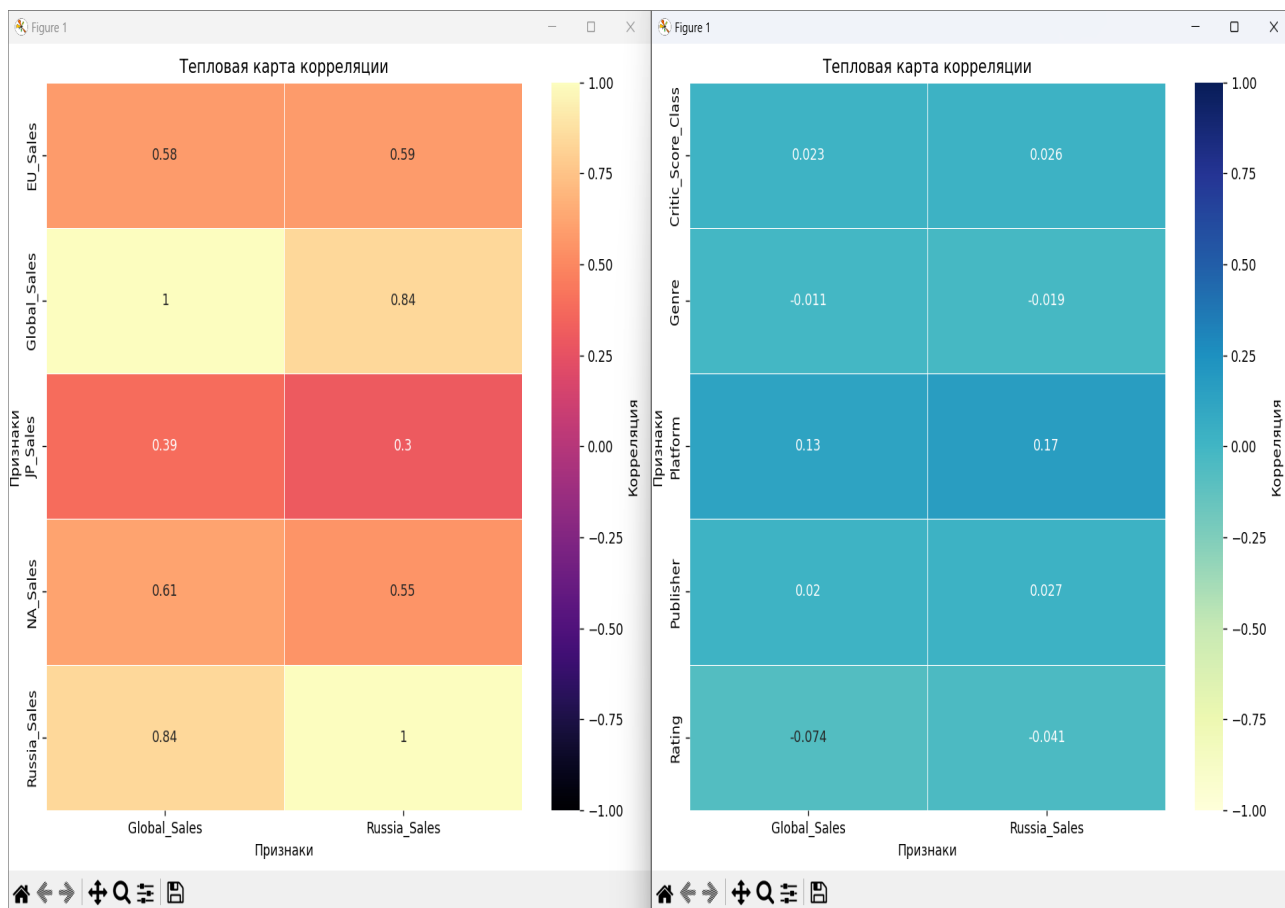


Рисунок 2.9 – Тепловые карты корреляции

2.3 Обучение моделей

Обучим несколько моделей для сравнения:

- линейная регрессия Logitom с автоподбором параметров для предсказания Global_Sales;
- нейросеть Logitom с подбором параметров для предсказания Global_Sales;
- линейная регрессия Logitom с Ridge-регуляризацией для предсказания Global_Sales;
- нейросеть Logitom с двумя скрытыми слоями по 10 нейронов соответственно для предсказания Global_Sales;

- линейная регрессия Loginom для предсказания Russia_Sales;
- нейросеть Loginom для предсказания Russia_Sales;
- несколько различных моделей на Python для предсказания Global_Sales;
- несколько различных моделей на Python для предсказания Russia_Sales.

Объявление моделей представлено на Рисунке 2.10.

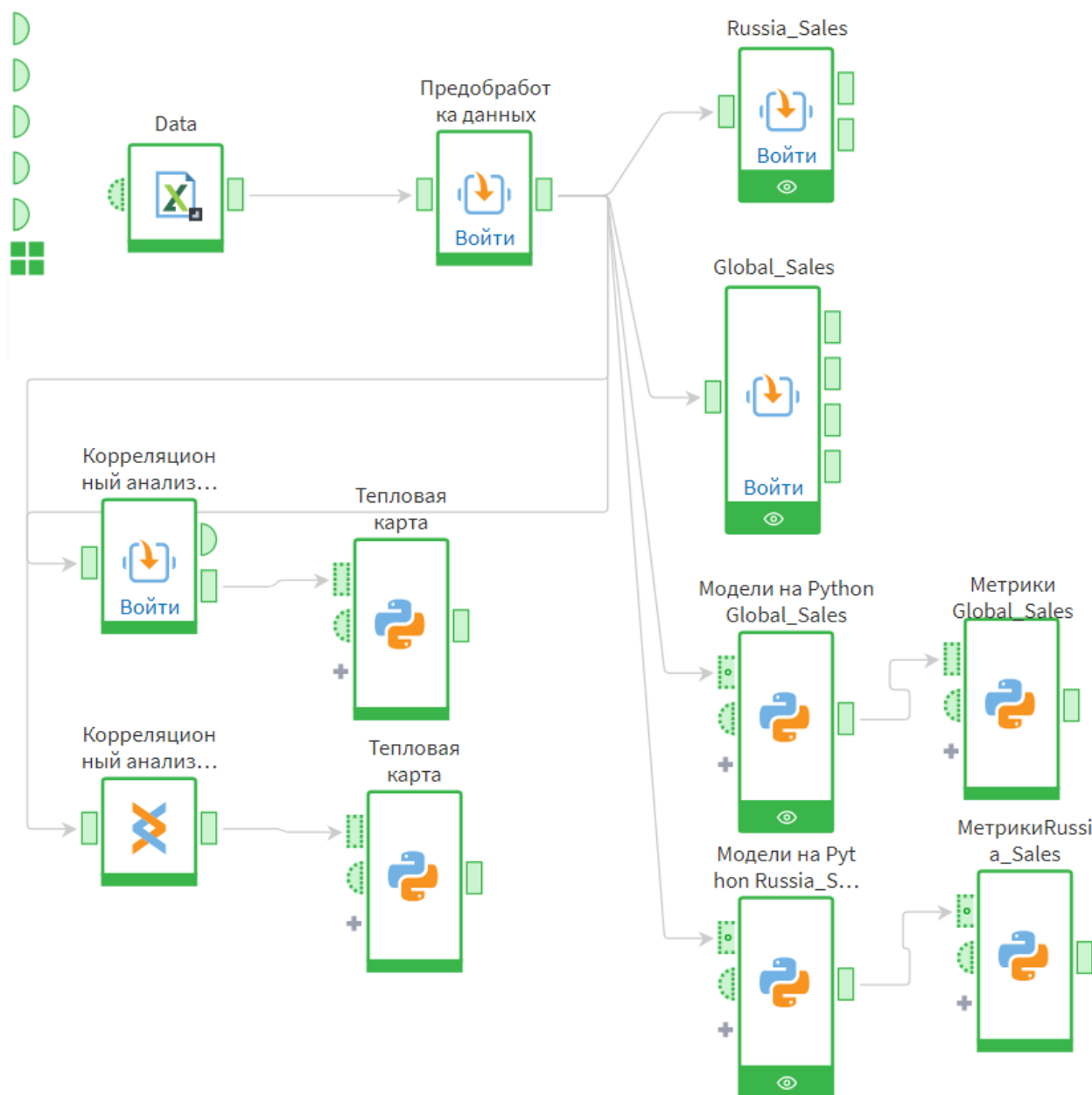


Рисунок 2.10 – Все регрессионные модели

Настройки Loginom моделей без автоподбора представлены на Рисунках 2.11 – 2.14.

Настройка линейной регрессии

Автоматическая настройка ☐

Приоритет автоматической настройки

Отбор факторов и защита от переобучения LASSO - Регрессия Лассо

Настройки приоритетов

Приоритет точность/скорость

Приоритет точные/недостоверные данные

Приоритет меньше/больше факторов

Денормализовать коэффициенты модели ☐

Использовать детальные настройки ☒

Показывать коэффициенты опорных категорий ☒

Рисунок 2.11 – Настройки линейной регрессии для предсказания Russia_Sales

Настройка параметров нейросети

Структура нейросети

Количество скрытых слоев Один скрытый слой

Количество нейронов в первом скрытом слое 10

Количество нейронов во втором скрытом слое 10

Ограничение на значение выходов Нет

Нижняя граница 0

Верхняя граница 1

Параметры обучения

Количество рестартов 10

Степень регуляризации 0

Продолжить обучение ☐

Критерии останова

Порог минимального изменения весов 0,005

Максимальное количество эпох 0

Рисунок 2.12 – Настройки нейросети для предсказания Russia_Sales

Рисунок 2.13 – Настройки линейной регрессии для предсказания Global Sales

Рисунок 2.14 – Настройки нейросети для предсказания Global Sales

Код модулей Python для обучения моделей представлен на Рисунках 2.15-2.16.

```
import builtin_data
from builtin_data import InputTable, InputTables, InputVariables, OutputTable, DataType, DataKind, UsageType
import numpy as np, pandas as pd
from builtin.pandas.utils import to_data_frame, prepare_compatible_table, fill_table
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor, GradientBoostingRegressor, StackingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.metrics import r2_score
import numpy as np
from sklearn.preprocessing import LabelEncoder

if InputTable:
    # Создать pd.DataFrame по входному набору №1
    data = to_data_frame(InputTable)
# Загрузка и подготовка данных
for column in data.columns:
    if data[column].dtype == 'object': # Если тип данных - строковый
        if data[column].nunique() <= 10:
            # One-Hot Encoding для признаков с небольшим количеством уникальных значений
            data = pd.get_dummies(data, columns=[column], drop_first=True)
        else:
            # Label Encoding для признаков с большим количеством уникальных значений
            le = LabelEncoder()
            data[column] = le.fit_transform(data[column])
X, y = data.drop(columns=['Global_Sales'], data['Global_Sales'])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Базовые модели для ансамбля
model_1 = RandomForestRegressor(n_estimators=50, random_state=42)
model_2 = GradientBoostingRegressor(n_estimators=50, random_state=42)
model_3 = SVR(kernel='rbf', C=1.0)
```

Рисунок 2.15 – Обучение моделей Python часть 1


```

bagging_model = BaggingRegressor(estimator=model_1, n_estimators=10, random_state=42)
bagging_model.fit(X_train, y_train)
bagging_pred = bagging_model.predict(X_test)
bagging_r2 = r2_score(y_test, bagging_pred)
print("Бэггинг R2:", bagging_r2)

stacking_model = StackingRegressor(
    estimators=[('rf', model_1), ('gb', model_2), ('svr', model_3)],
    final_estimator=LinearRegression()
)
stacking_model.fit(X_train, y_train)
stacking_pred = stacking_model.predict(X_test)
stacking_r2 = r2_score(y_test, stacking_pred)
print("Стэкинг R2:", stacking_r2)

def average_predictions(models, X):
    preds = [model.predict(X) for model in models]
    return np.mean(preds, axis=0)

model_1.fit(X_train, y_train)
model_2.fit(X_train, y_train)
model_3.fit(X_train, y_train)
average_pred = average_predictions([model_1, model_2, model_3], X_test)
average_r2 = r2_score(y_test, average_pred)
print("Усреднение предсказаний R2:", average_r2)
output_frame = pd.DataFrame({
    "Actual_Global_Sales": y,
    "Predict_Bagging": bagging_model.predict(X),
    "Predict_Stacking": stacking_model.predict(X),
    "Predict_Average": average_predictions([model_1, model_2, model_3], X)
})

if isinstance(OutputTable, builtin_data.ConfigurableOutputTableClass):
    prepare_compatible_table(OutputTable, output_frame, with_index=False)
fill_table(OutputTable, output_frame, with_index=False)

```

Рисунок 2.16 – Обучение моделей Python часть 2

Визуализируем предсказания моделей. Графики представлены на Рисунках 2.17-2.20.

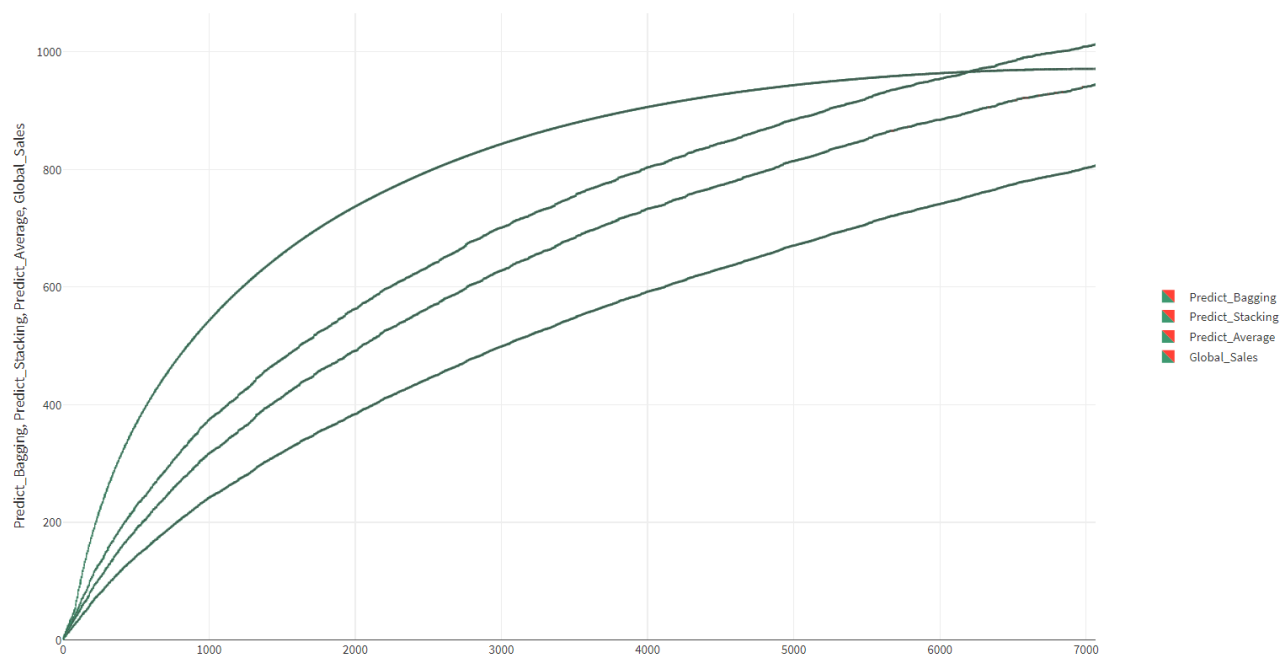


Рисунок 2.17 – Кривые обучения моделей для Global_Sales

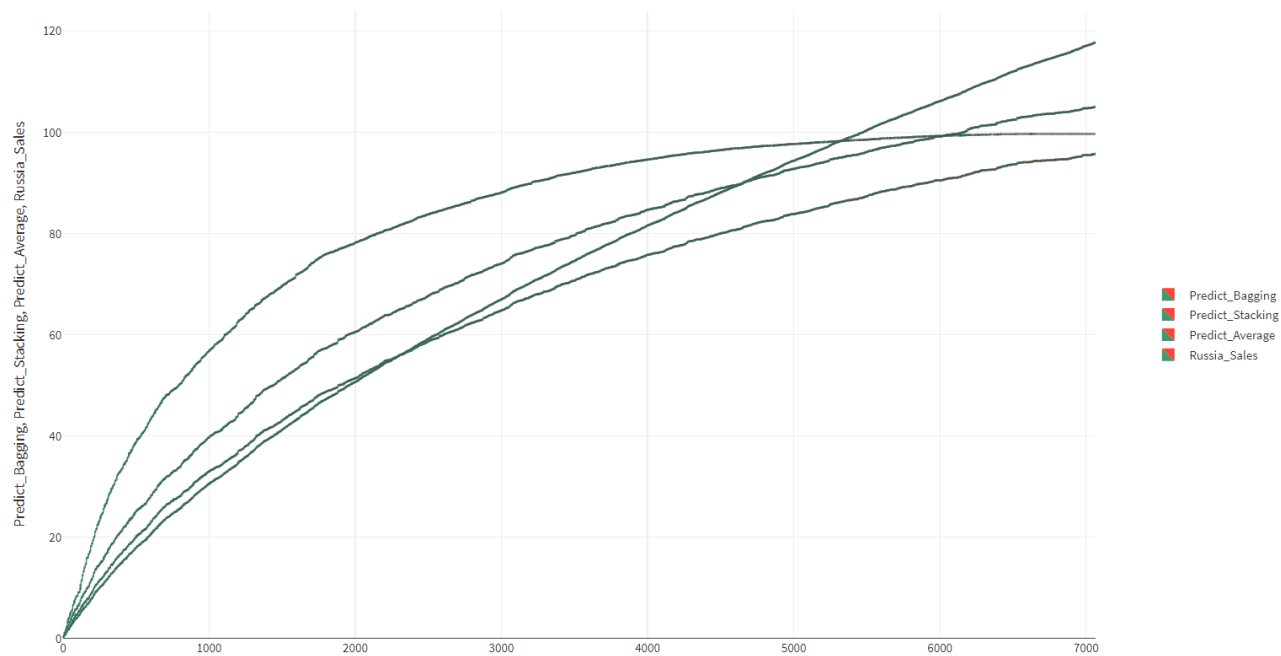


Рисунок 2.18 – Кривые обучения моделей для Russia_Sales

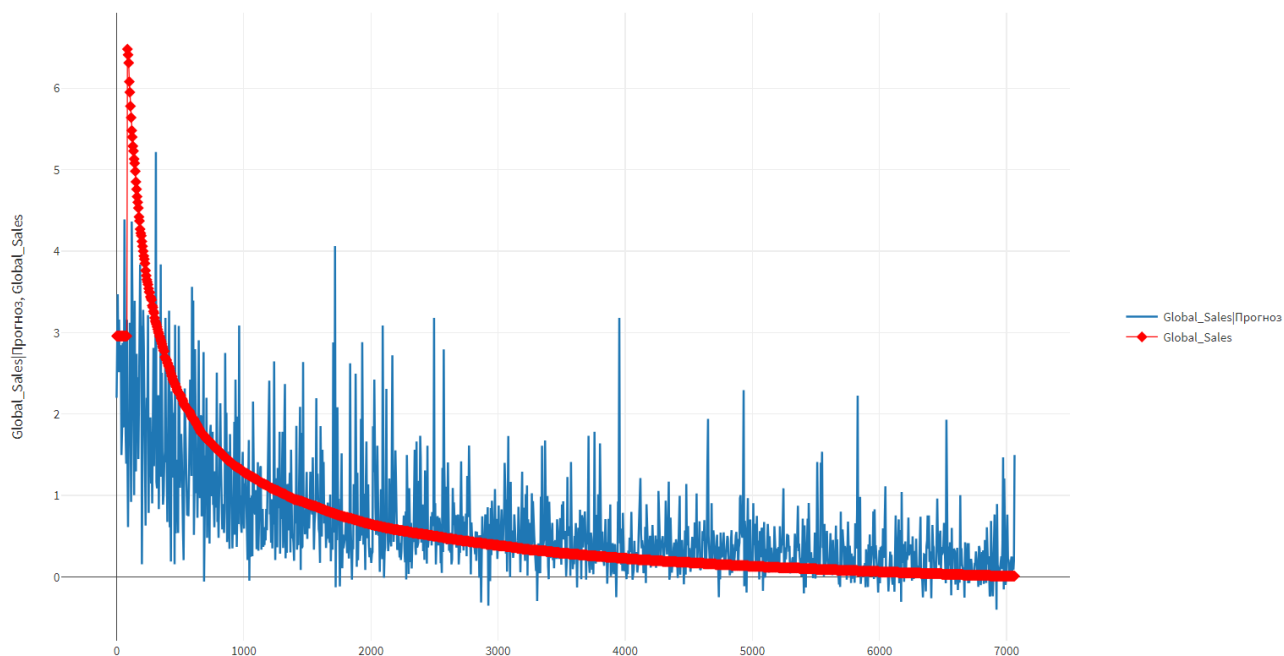


Рисунок 2.19 – График предсказаний лучшей Loginom модели для Global_Sales

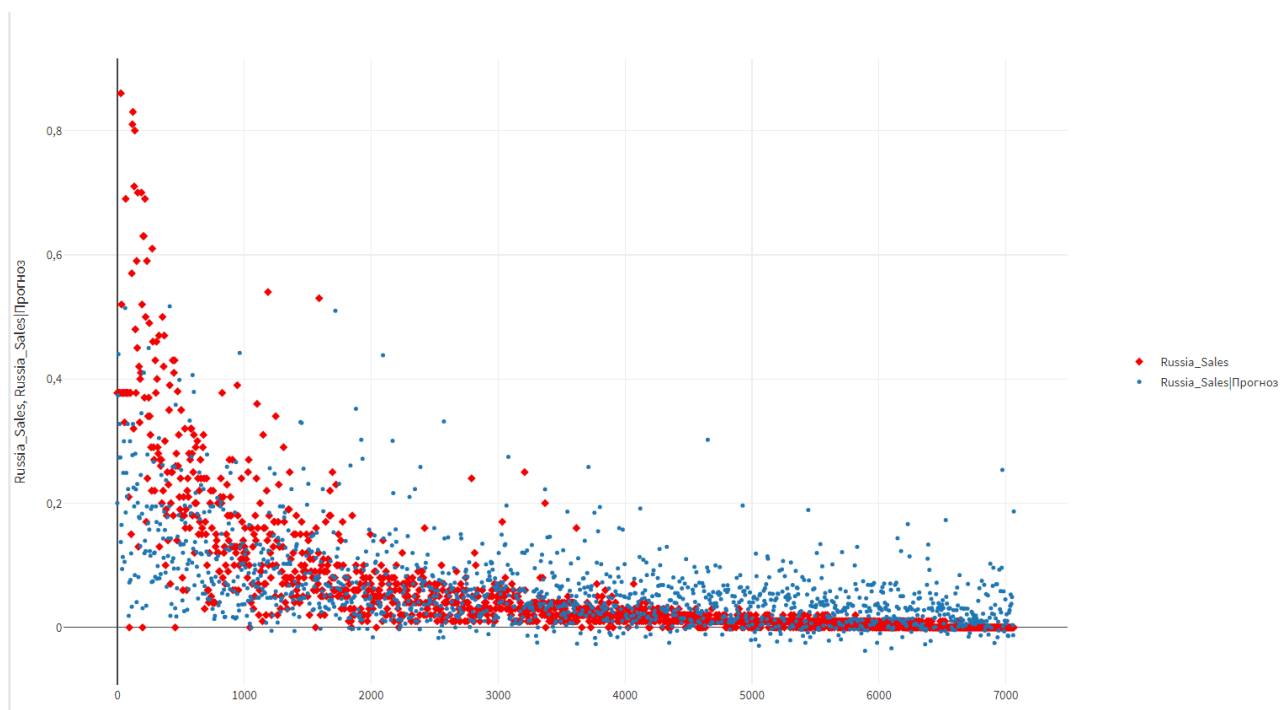


Рисунок 2.20 – График предсказаний лучшей Loginom модели для Russia_Sales

Проанализируем метрики всех моделей, объединив их в одну таблицу. Полученная таблица представлена на Рисунке 2.21.

Метрики • Быстрый просмотр					
Выходной набор данных					
#	ab Название модели	90 MAE	90 MSE	90 r2	
1	Нейросеть Loginom Russia_Sales	0,06	0,01	0,28	
2	Линейная регрессия Loginom Russia_Sales	0,05	0,01	0,38	
3	Линейная регрессия 1 Loginom Global_Sales	0,53	0,70	0,22	
4	Нейросеть 1 Loginom Global_Sales	0,42	0,48	0,46	
5	Нейросеть 2 Loginom Global_Sales	0,48	0,62	0,30	
6	Линейная регрессия 2 Loginom Global_Sales	0,53	0,69	0,22	
7	Predict_Bagging Global_Sales	0,36	0,36	0,59	
8	Predict_Stacking Global_Sales	0,39	0,43	0,52	
9	Predict_Average Global_Sales	0,39	0,50	0,44	
10	Predict_Bagging Russia_Sales	0,04	0,01	0,59	
11	Predict_Stacking Russia_Sales	0,04	0,01	0,51	
12	Predict_Average Russia_Sales	0,06	0,01	0,43	

Рисунок 2.21 – Таблица метрик всех моделей

Из таблицы видно, что лучше всего справились бэггинг модели, судя по метрике R2_score, но при предсказании продаж в России среднеквадратичная ошибка меньше, чем при предсказании продаж в мире.

ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы была проведена исследовательская работа по задаче линейной регрессии, как в её классической форме, так и с использованием нейросетей. Исследование включало теоретическую часть, охватывающую основы регрессионного анализа, различные типы регрессий, метрики оценки качества моделей и особенности работы нейросетей, а также практическую часть, включающую предобработку данных, обучение моделей и анализ результатов.

Практическая часть работы была посвящена анализу рынка видеоигр в России и мире. Для этого была использована база данных "Videogames_Sale_Data", содержащая информацию о продажах видеоигр по различным платформам, жанрам и издателям. Была проведена предобработка данных и корреляционный анализ.

Для обучения моделей были использованы как классические методы, так и нейросетевые модели с различной архитектурой. Модели были обучены на данных о глобальных и региональных продажах видеоигр. В результате анализа метрик качества предсказаний было выявлено, что бэггинг модели показал наилучшие результаты по метрике R^2_score .

Таким образом, можно сделать вывод, что выбор метода регрессии должен зависеть от специфики задачи и данных. Для линейных зависимостей и качественных данных классические методы регрессии являются предпочтительными, тогда как для сложных нелинейных зависимостей и больших наборов данных нейросетевые модели демонстрируют лучшие результаты.

Данная работа подтвердила важность выбора подходящего метода регрессии в зависимости от задачи и данных, а также показала перспективы применения нейросетевых моделей для решения задач регрессии.

ИНФОРМАЦИОННЫЕ ИСТОЧНИКИ

Теоретическая часть

- 1.1. Гельман Э., Хилл Дж., Вехтари А. Регрессия: теория и практика/ Гельман Э., Хилл Дж., Вехтари А. — Москва: Лань, 2022. — 748с.
- 1.2. Паттерсон Дж., Гибсон А. Глубокое обучение с точки зрения практика/ Паттерсон Дж., Гибсон А.— Москва: Лань, 2018. — 418 с.
- 1.3. Чио К., Фримэн Д. Машинное обучение и безопасность/ Чио К., Фримэн Д. — Москва: Лань, 2020. — 388 с.
- 1.4. Евстафьев В. А., Тюков М. А. Искусственный интеллект и нейросети: практика применения в рекламе: Учебное пособие - Издательство "Дашков и К", 2023 – 426 с.
- 1.5. Душкин Р. В. Искусственный интеллект - Издательство "ДМК Пресс", 2019 – 280с.

Практическая часть

- 2.1. Мясников В. В. Распознавание образов и машинное обучение. Основные подходы: учебное пособие — Москва: Лань, 2023. — 196 с.
- 2.2. Сорокин А. Б., Железняк Л. М., Семенов Р. Э. Технологии обучения: регрессия и прогнозирование: Учебно-методическое пособие/ Сорокин А. Б., Железняк Л. М., Семенов Р. Э. — Москва: Лань, 2022. — 106 с.
- 2.3. Сидняев Н.И., Храпов П.В. Нейросети и нейроматематика: учебное пособие - Московский государственный технический университет имени Н.Э. Баумана, 2016 – 83 с.

ПРИЛОЖЕНИЕ

Приложение А — Графический материал.

Приложение А

На рисунке 3.1 представлен сценарий проекта в аналитической платформе Loginom, полученный в результате выполнения данной курсовой работы.

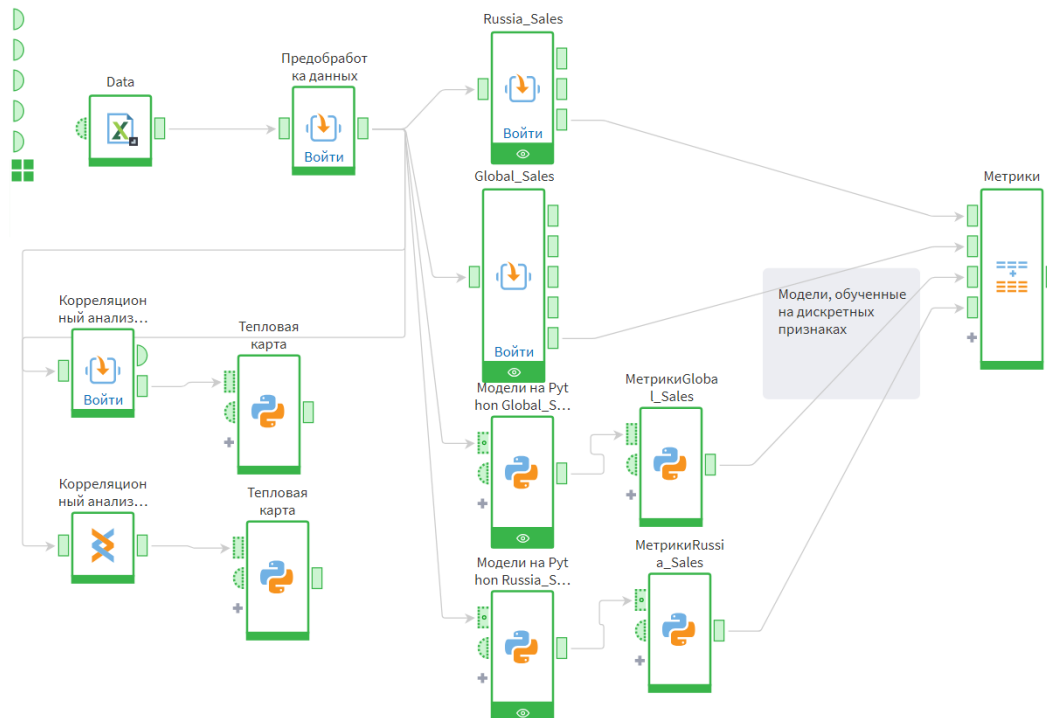


Рисунок 3.1 — Сценарий проекта в аналитической платформе Loginom

Этапы выполнения сценария:

1. Импорт исходных данных (Excel файлы).
2. Предобработка исходных данных.
3. Корреляционно-регрессионный анализ.
4. Обучение моделей.
5. Сравнение метрик.